Github: https://github.com/JydnMrsh/DPSM-JaydenMarsh-23217931

Enterprise Systems Analysis, Design, and Modelling Project-I

Jayden Marsh
23217931

Task 1: Organisational and System Analysis

Company Overview:

Company: Bunning Warehouse.

Bunnings Warehouse is a large hardware store based in New Zealand and Australia. They have over three hundred stores, more than fifty thousand employees, and operate multiple business-critical systems. Bunnings Warehouse is used by both everyday customers and commercial tradespeople.

Departmental Structure and Responsibilities.

Bunnings Warehouse has a very traditional retail store departmental structure, with logistics, warehousing, retail, HR, and more. Where most of their departments use an inventory management system on a daily basis.

Departments and their responsibilities:

- Logistics & Supply Chain:
    - Manages product movement from suppliers to stores.
    - Overseas transport.
    - Delivery and delivery scheduling.
- Warehouse Operations:
    - Receives, stores, and dispatches items.
    - Maintains stock levels.
- Retail Operations:
    - Connects warehouse stock to in-store stock.
    - Restocks shelves.
- IT Department:
    - Develops and maintains IT systems.
    - Integrates new or improved systems.
- Managers/Finance Department:
    - Tracks stock costs and revenue.
    - Manage orders.
- Human Resources:
    - Manages staff, training, and workplace safety.
    - Manages the hiring process.

Identification of Stakeholders and User Roles.

Stakeholders:

- Internal
    - Warehouse staff
    - Retail staff
    - Store managers
    - Senior managers
    - IT staff
    - Parent company (Wesfarmers)

- External
    - Customers
    - Suppliers
    - Logistic/Transport companies
    - Investors

After identifying the internal and external stakeholders, I interviewed/discussed with them to find out what their user roles are when dealing with an inventory management system so I can understand who would be most impacted by a change of inventory management system.

Warehouse Staff:
The warehouse staff interact with their inventory management system throughout the day. They use it to monitor stock, update inventory levels, and record receipts of goods from suppliers. They ensure the physical stock matches what the system says. The warehouse staff's access to the system lets them update stock and enter receipts, but they cannot change prices or make orders from suppliers.

Retail Staff:
The retail staff interact with their inventory management system throughout the day. They use it to check the availability of products to assist customers, and the cashiers use it to update stock levels after sales. They say their system doesn't let them know if stock is in the warehouse or on shelves, just whether it's in stock or not, which makes it harder to keep shelves full. The retail staff's access lets them update prices, but they cannot directly update stock levels, only through the use of the checkout machines after scanning items.

Store Managers:
Store managers use the system daily at a higher level to oversee performance and operations. They use it to monitor stock levels and see what products are selling fast or slow. They also use it to handle stock discrepancies, approve adjustments, and to make sure both retail and warehouse operations are in sync. The store managers have elevated access to the system, and they can change prices, update stock levels, and make orders if the warehouse staff ask them to.

Senior Managers:
Senior managers don't use the inventory management system often. They are usually at an office, not in the store, and communicate with the store manager if they need any information about the inventory. They don't have direct access to the system, but they can access anything they need through the store manager.

IT Department:
The IT department only interacts with the inventory management system a few times a week. They are responsible for maintaining the system, updating the system, and integrating with other systems. The inventory system only needs to be accessed by them a few times a week because system updates or issues don't happen daily. The IT department has admin access to the system, they can update prices or stock levels but they only have to do so very rarely. They need admin privileges for the backend side of things, like updating the system or managing the access levels of other users/departments.

Human Resources:
HR don't interact with the inventory system directly. They manage staff issues, and the hiring and training process. They have a good understanding of the inventory system as they train staff to use it, but they don't use it themselves. The HR department doesn't have access to the system. They can't see stock levels, can't update prices, and can't make orders.

Customers:
Customers interact with the system indirectly and directly. Customers indirectly update stock levels by buying items and using self-checkout machines. Some customers also directly access the system by looking at stock levels or availability through our online store. Customers have frontend access to the system only.

Suppliers:
Suppliers interact indirectly with the system by providing stock, which will get processed through the system. Suppliers don't have direct access to the system, they can't see stock levels and can't make orders.

Investors:
Investors don't interact with the inventory system, however they are still stakeholders who are affected by the performance of the company, which could be affected by the performance of the inventory system.

Analysis:
Bunnings has a good legacy system in place that does the basics of what they need for an inventory system, but I think some improvements could be made. I think the main problem is that the retail team and warehousing team have to manually communicate with each other about where stock is located in the store, and they have to manually check to see if stock is low and then go and ask the store manager to order more. I think Bunnings could benefit from a more automated inventory management system.

I think Bunnings could benefit from a more automated inventory management system, where it can track the location of items, give automated alerts of low stock levels, be able to send order requests to the store manager, and see what stock has already been ordered/is on the way.

System Scope, Boundaries, and Assumptions.

System Scope:

The improved inventory management system for Bunnings Warehouse is designed to manage and track real-time stock levels, prices, locations, and order requests across the warehouse and retail departments, providing all departments with more information about what the stock is doing so the departments can act more as one seamless operation rather than separate ones.

The system will:

- Track inventory levels.
- Update inventory levels.
- Record incoming stock.
- Manage stock transfers between the warehouse and the retail departments.
- Track the location of items.
- Update the location of items.
- Send order requests to store managers for approval.

The system involves multiple departments, including retail, warehousing, logistics, and managers/finance. The system supports multiple user roles, such as warehouse staff, retail staff, managers, IT/admins, suppliers, and customers.

System Boundaries:

The system boundary is limited to inventory related operations and doesn't deal with functionality outside of stock management.

The system will NOT:

- Manage employee payroll
- Manage HR processes
- Process payments
- Be able to make orders from suppliers, it will only have the ability to request an order of stock to the store manager (who will use a different system to place the order).

Assumptions:

These assumptions were made when defining the system:

- All Bunnings stores use an inventory management system.
- Stock updates need to happen in real-time in all departments.

- Retail sales systems, eg. self-checkout machines and cashiers, are integrated with the inventory system to automatically update stock when items are purchased.
- Customers are allowed to see stock levels and availability through the online website.
- Bunnings already has other systems for communicating and making orders with suppliers.
- Bunnings has both the retail department and warehouse department located in the same store/location, so they can store stock on site.

Task 2: Requirements Engineering

Functional Requirements:

Functional requirements are what the system does, they define the features, operations, and behaviours that the system must be able to do to function.

The inventory system must have the following functional requirements:

Inventory Tracking:

1. The system will track stock levels across the warehouse and retail departments.
- Relates to the stakeholders: Warehouse staff, Retail staff

2. The system will automatically update the stock level when items are received or sold.
- Relates to the stakeholders: Warehouse staff, Retail staff, Customers

3. The system will allow all users to see stock levels.
- Relates to the stakeholders: Warehouse staff, Retail staff, Store managers, Customers

Item Location Tracking:

4. The system will track the physical location of items.
- Relates to the stakeholders: Warehouse staff, Retail staff

5. The system will automatically update the location of items if they are moved from the warehouse into the retail department.
- Relates to the stakeholders: Warehouse staff, Retail staff

6. The system will allow authorised users to view the location of items.
- Relates to the stakeholders: Warehouse staff, Retail staff, Store managers

Stock Alerts:

7. The system will detect items that have low stock and alert users.
- Relates to the stakeholders: Warehouse staff, Retail staff

Order Requests:

8. The system will let the user generate an order request for any item and send it to the store manager.
- Relates to the stakeholders: Warehouse staff

9. The system will allow store managers to approve or deny the order requests.

- Relates to the stakeholders: Store managers

10. The system will show the status of order requests.
- Relates to the stakeholders: Warehouse staff

Scalability:

11. The system will let you add new items and automatically assign it a new item ID.
- Relates to the stakeholders: Warehouse staff

Non-Functional Requirements:

Non-functional requirements are how the system performs the functions, they define the quality and constraints of the system such as performance, reliability, security, and user-friendliness.

The inventory system must have the following non-functional requirements:

Performance:

1. The system will update stock levels within 1 second.
- Relates to the stakeholders: Warehouse staff, Retail staff, Customers

2. The system will be up 24/7 in case customers are browsing the stock level of an item through the Bunnings website outside of store opening hours.
- Relates to the stakeholders: Customers

Security:

3. The system will require you to login with a username and password.
- Relates to the stakeholders: All users except customers as they use the website to view stock levels.

4. The system will have different levels of access based on your role (eg. Admin, Manager, Warehouse, Retail, Customer).
- Relates to the stakeholders: All users

Usability:

5. The system will have a GUI interface that is easy to use.
- Relates to the stakeholders: All users except customers as they use the website to view stock levels.

Identifying Constraints:

To keep the project scope small and for the inventory system to integrate well, Bunnings must already have the following infrastructure:
- A website for users to browse products that updates the stock levels based off of a database (that my inventory system will update).
- Barcode scanners that can identify an item's ID and run code or a program (which will use my inventory system for updating item attributes, like location or stock level).
- Existing servers to run the inventory system.

Task 3: UML Requirements Modelling

Identifying Each Actor's Needs:

Actors:
Warehouse Staff
- LogIn
- UpdateStockLevel
- TransferStockToRetail
- ViewStockLevel
- ViewItemLocation
- GenerateOrderRequest
- ViewOrderStatus

Retail Staff
- LogIn
- ViewStockLevel
- UpdateStockLevel
- TransferStockToWarehouse
- ViewItemLocation
- ReceiveLowStockAlert

Store Manager
- LogIn
- ViewStockLevel
- ViewItemLocation
- ApproveOrderRequest
- RejectOrderRequest

Customer
- CheckItemAvailability

IT Staff
- LogIn
- CreateUser
- ManageUserPermission

Identifying Relationships:

Any action that requires a logged in user should include LogIn:
UpdateStockLevel <<include>> LogIn
TransferStockToRetail <<include>> LogIn
TransferStockToWarehouse <<include>> LogIn
ViewStockLevel <<include>> LogIn
ViewItemLocation <<include>> LogIn
GenerateOrderRequest <<include>> LogIn
ViewOrderStatus <<include>> LogIn

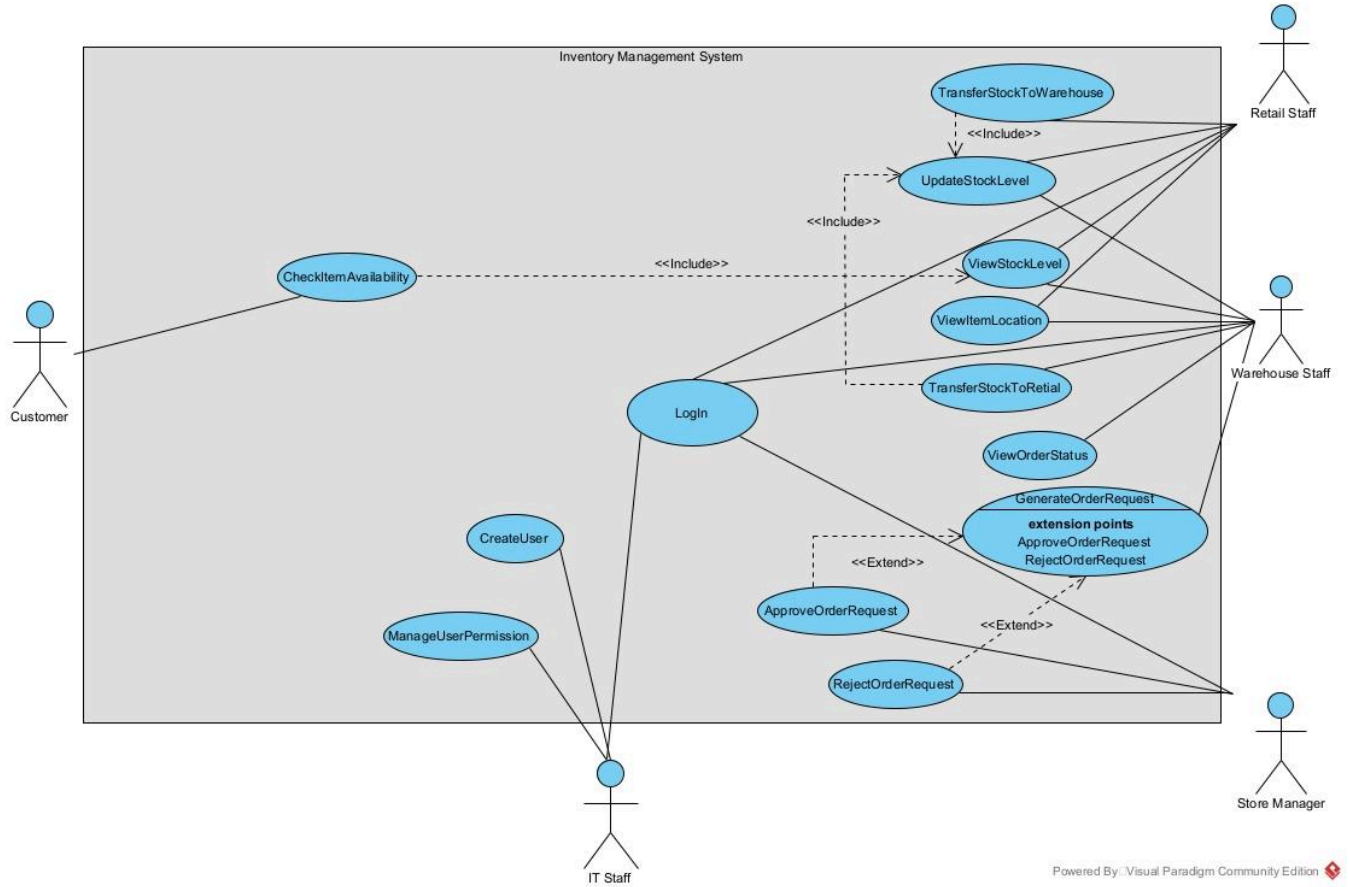
ApproveOrderRequest <<include>> LogIn
RejectOrderRequest <<include>> LogIn
CreateUser <<include>> LogIn
ManageUserPermission <<include>> LogIn

Transfers of stock will need to update the stock level should include UpdateStockLevel:
TransferStockToRetail <<include>> UpdateStockLevel
TransferStockToWarehouse <<include>> UpdateStockLevel

Before approving or rejecting order requests and order must exist first:
ApproveOrderRequest <<extend>> GenerateOrderRequest
RejectOrderRequest <<extend>> GenerateOrderRequest

A customer viewing item availability through the website will require ViewStockLevel:
CheckItemAvailability <<include>> ViewStockLevel

Use Case Diagram:



Use Case Descriptions:

Use Case 1:	Update Stock Level

Main Actor:	Warehouse Staff, Retail Staff
Description:	Updates the stock level of an item after receiving or transferring stock.
Preconditions:	- User is logged in and has permission - Item exists in the system
Postconditions:	- Stock level gets updated
Main Flow:	1. User selects item 2. User enters new amount or adds/removes an amount 3. System updates stock level 4. System confirms update
Alternate Flow:	1. Invalid quantity 2. System displays an error message
Extension Points:	Low Stock Alert extends Update Stock Level if the stock of an item goes below a threshold

Use Case 2:	Transfer Stock to Retail
Main Actor:	Warehouse Staff
Description:	Updates the location of an item from warehouse to retail
Preconditions:	- User is logged in and has permission - Item exists in the system - There is enough stock in the warehouse
Postconditions:	- Warehouse stock decreases - Retail stock increases - Item location gets updated
Main Flow:	1. User selects item 2. User enters amount to transfer 3. System checks there's enough stock 4. System updates stock levels 5. System updates item locations 6. System confirms the transfer
Alternate Flow:	1. Not enough stock 2. System displays error message
Extension Points:	N/A

Use Case 3:	Generate Order Request
Main Actor:	Warehouse Staff
Description:	Creates and sends an order request to the store manager
Preconditions:	- User is logged in and has permission - Item exists in the system
Postconditions:	- Order request is created with a pending status - Sends order to the store manager
Main Flow:	1. User selects item 2. User enters the amount of stock they want to request 3. System creates order request 4. System sends order request to store manager 5. System confirms the request was sent
Alternate Flow:	1. Invalid quantity 2. System displays and error message
Extension Points:	N/A

Use Case 4:	Approve/Reject Order Request
Main Actor:	Store Manager
Description:	Approves or rejects the pending order request from the warehouse staff
Preconditions:	- User is logged in and has permission - Order request has been sent and is pending
Postconditions:	- Order status gets changed to approved
Main Flow:	1. Manager views the pending order request 2. Manager approves the request 3. The order is removed from the store manager's requests list 4. System updates the status to approved 5. System confirms you approved the order
Alternate Flow:	1. Manager rejects the request 2. Status is updated to rejected
Extension Points:	Aprrovinging/Rejecting Order Request extends Generate Order Request

Use Case 5:	View Stock Level
Main Actor:	Warehouse Staff, Retail Staff, Store Manager
Description:	Displays the current stock levels of items
Preconditions:	- User is logged in and has permission - Items exist
Postconditions:	- Stock levels are displayed
Main Flow:	1. User is on the main screen 2. System displays the stock levels of existing items
Alternate Flow:	1. No items exist 2. Blank display
Extension Points:	N/A

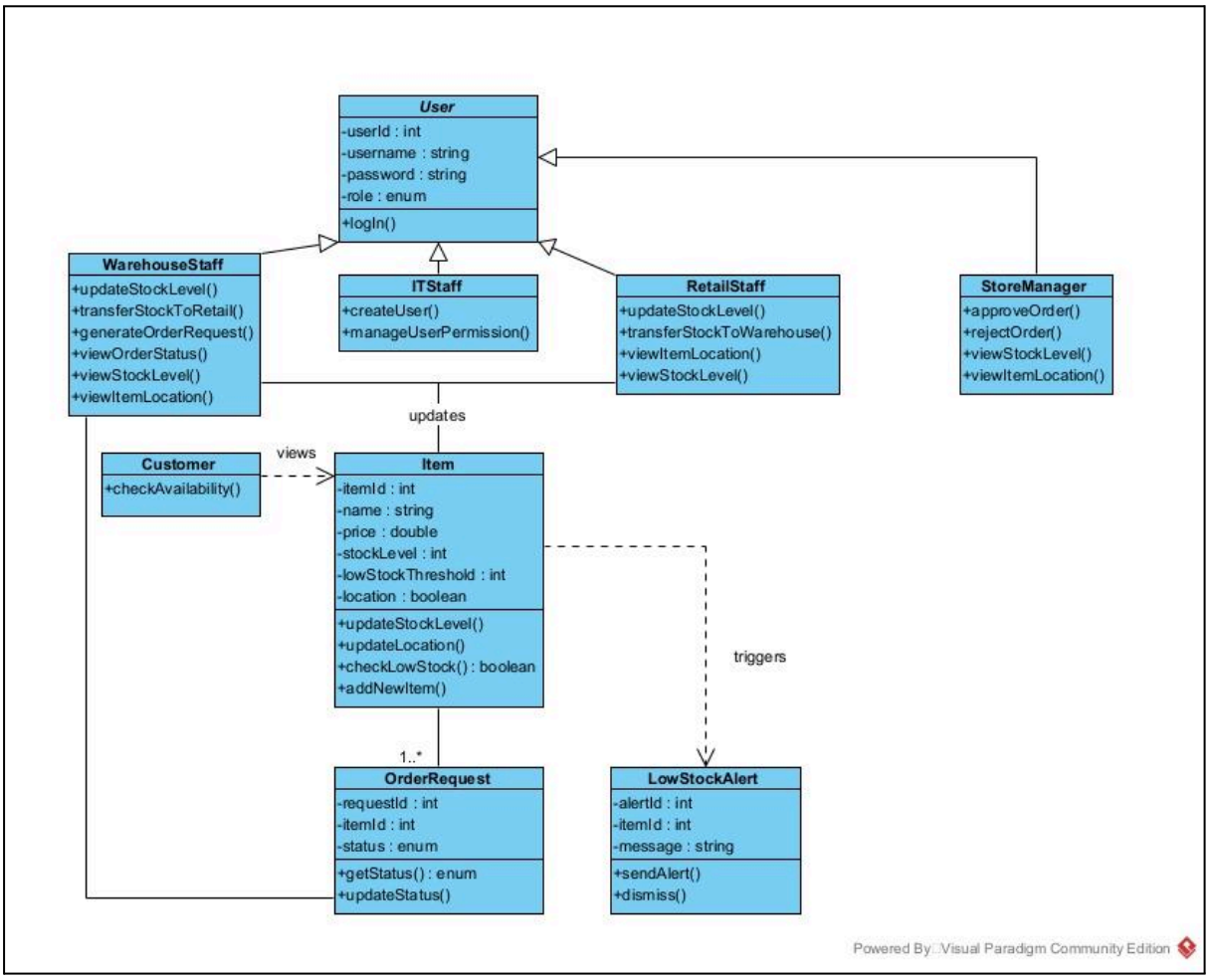
Use Case 6:	View Item Location
Main Actor:	Warehouse Staff, Retail Staff, Store Manager
Description:	Displays the current location of items
Preconditions:	- User is logged in and has permission - Items exist
Postconditions:	- Item location is displayed
Main Flow:	1. User is on the main screen 2. Items exist
Alternate Flow:	1. Location not set 2. No location displayed
Extension Points:	N/A

Use Case 7:	Log In
Main Actor:	Warehouse Staff, Retail Staff, Store Manager, IT Staff
Description:	Lets you log in with a username and password and gives you different access based on your role.
Preconditions:	- User is on a machine that has the system installed

Postconditions:	- Lets the user access the system with varying access
Main Flow:	1. User opens the system 2. User has to enter a username and password 3. User enters their correct username and password that is linked to their role 4. User gains access to the system
Alternate Flow:	1. Incorrect username or password 2. System displays error message
Extension Points:	N/A

Task 4: Initial Structural Design and Implementation Planning

Class Diagram:



Class Planning:

User is an abstract class with shared attributes userId, username, password, and role, along with the LogIn() method.WarehouseStaff, RetailStaff, StoreManager, and ITStaff are inheriting from User with generalisation as they are unique types of a User. Customer is separate because they access the system externally through the website without logging into the system. The role attribute from User, and the status attribute from the OrderRequest will be implemented as C# enums. The Item class will have the main inventory data and methods.

Identification of Design Weaknesses:
Duplicate methods: UpdateStockLevel() and ViewStockLevel() are in both WarehouseStaff and RetailStaff, and ViewStockLevel() and ViewItemLocation() are in WarehouseStaff, RetailStaff, and StoreManager.

OrderRequest has no way to tell who to send it to: The OrderRequest class has no attribute of where to send the request. For my implementation, it will always be to the store manager (and I'm assuming there is only one store manager) so I will hard-code it to send to the store manager, but it doesn't have good scalability, eg. if you had multiple store managers, or wanted to send the request somewhere else.

Item class is doing too much: The Item class handles stock level updates, location updates, low stock checks, and adding new items. The Item class might be responsible for too many things at once.

No interface for permissions: Different roles have different levels of access, but this is only implied by which subclasses have which methods. There is no proper interface or access control, meaning the permission logic will be scattered all over the code, eg. each time you press a button it will check what role the user has to determine if it runs.